SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 1
Case Study

COURSE NAME: DEEP LEARNING **COURSE CODE: MLA0408**

COURSE OUTCOME COVERED

CO3: Analyze and apply convolutional neural network architectures, including convolutional layers, filters, parameter sharing, regularization, AlexNet and ResNet, to develop solutions for image classification and real-world computer vision applications. (L4)

CO Weightage: 25%

Assessment Tool Weightage: 25%

QUESTIONS: **Total Marks: 25**

Case Study 1: CNN for Automated Medical Image Classification

A healthcare organization wants to develop a deep learning system to automatically classify chest X-ray images into **normal** and **disease-affected** categories. The dataset contains thousands of X-ray images with variations in size, brightness and image quality. A CNN model is proposed because it can automatically learn important visual features such as edges, textures, and abnormal regions without manually designing features.

Question:

1. Design and explain a suitable **CNN architecture** for this medical image classification system.
2. Describe the **motivation for using CNNs** and explain the role of **convolutional layers, filters, pooling layers, and fully connected layers** in the proposed architecture.
3. Explain how **parameter sharing** reduces the number of trainable parameters compared with a fully connected network.
4. Discuss suitable **regularization techniques such as dropout, data augmentation and batch normalization** to reduce overfitting.
5. Finally, explain whether **AlexNet or ResNet** would be more appropriate for this application and justify your choice based on feature learning, network depth, computational requirements and classification performance.

Case Study 2: CNN-Based Autonomous Vehicle Object Recognition

An autonomous vehicle company is developing a vision system that must identify **cars, pedestrians, traffic signs, bicycles, and road obstacles** from images captured by cameras mounted on the vehicle. The system must recognize objects under different lighting conditions, viewing angles, distances and road environments. The development team is considering **AlexNet and ResNet** architectures for building the CNN-based recognition system.

Question:

1. Analyze how a **CNN architecture** can be designed for this autonomous vehicle application.
2. Explain the purpose of the **input, convolution, activation, pooling, and fully connected/output layers** and describe how convolutional **filters progressively learn low-level and high-level features**.
3. Explain the importance of **parameter sharing** in reducing computational complexity and improving translation-related feature detection.
4. Discuss the possible problem of **overfitting** and recommend suitable regularization methods.
5. Compare **AlexNet and ResNet** in terms of architecture, depth, parameter efficiency, training difficulty, vanishing-gradient problems and feature-learning capability.
6. Finally, recommend the most suitable architecture for the autonomous vehicle system and justify your decision based on **accuracy, computational cost and real-time application requirements**.

Code of Conduct:

I Arya.B.Nair 192424133 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Answers

CASE STUDY 1

1. Suitable CNN Architecture

A suitable architecture can be designed as follows:

Input Image → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Fully Connected Layer → Output Layer

1. Input Layer

The chest X-ray images can first be resized to a fixed size, such as **224 × 224 pixels**, so that all images have the same dimensions. The pixel values can also be normalized to improve the training process.

2. First Convolutional Layer

The first convolutional layer uses several small filters, such as **3 × 3 filters**, to scan the X-ray image. These filters learn simple features such as **edges, boundaries and basic intensity patterns**.

A **ReLU activation function** is applied after convolution to introduce non-linearity and allow the network to learn more complex patterns.

3. First Pooling Layer

A **max-pooling layer**, for example with a 2 × 2 window, reduces the spatial dimensions of the feature maps. This decreases computational requirements while retaining important features.

4. Deeper Convolutional Layers

Additional convolutional layers are added to learn increasingly complex features. The earlier layers may identify edges and textures, while deeper layers can learn patterns associated with **abnormal regions and disease-related structures** in the chest X-ray.

Pooling can again be applied after these convolutional blocks to progressively reduce the spatial size of the feature maps.

5. Fully Connected Layer

After the convolution and pooling stages, the resulting feature maps are flattened into a one-dimensional vector. A fully connected layer then combines the learned features and uses them to distinguish between normal and disease-affected X-rays.

Dropout can also be applied here to reduce overfitting.

6. Output Layer

Since this case has **two classes**, the final output layer can contain two outputs representing:

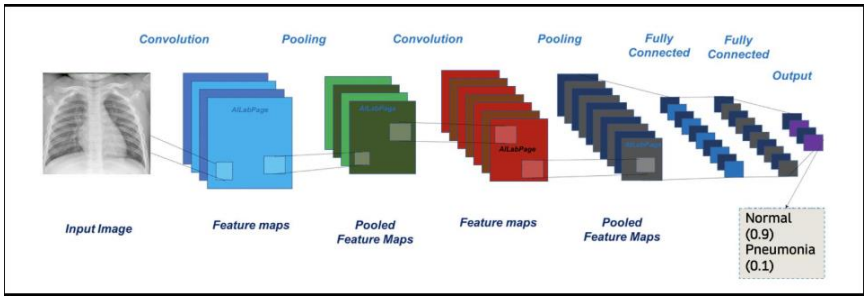
- **Normal**
- **Disease-affected**

A **softmax activation** can provide the probability of each class, and the class with the higher probability can be selected as the prediction.

Overall Architecture

Chest X-ray
↓
Preprocessing / Resizing
↓
Conv + ReLU
↓
Max Pooling
↓
Conv + ReLU
↓
Max Pooling
↓
Conv + ReLU
↓
Max Pooling
↓
Flatten
↓
Fully Connected + Dropout
↓
Softmax Output
↓
Normal / Disease-Affected

This architecture is appropriate because CNNs can automatically learn visual features from the X-ray images rather than requiring manually designed features, which is one of the main motivations given in the case study.



2. Motivation for Using CNNs

CNNs are well suited for chest X-ray classification because they can **automatically learn important visual features** directly from images. Unlike traditional approaches where features such as edges, textures, and abnormal regions have to be manually designed, a CNN learns these features during training. This makes CNNs particularly useful when working with thousands of medical images.

The different layers of the CNN perform different functions:

Convolutional Layers

Convolutional layers are responsible for extracting features from the X-ray images. They apply small filters across different regions of the image and generate **feature maps**.

The initial convolutional layers can learn simple patterns such as:

- Edges
- Lines
- Textures
- Intensity changes

As the network becomes deeper, the convolutional layers can combine these basic features to recognize more complex patterns, such as structures or regions that may indicate disease.

Filters

Filters, also called **kernels**, are small matrices that move across the input image. Each filter is trained to detect a particular type of feature.

For example, one filter may learn to detect horizontal edges while another may respond to a particular texture or shape. Multiple filters allow the CNN to extract different features from the same X-ray.

Pooling Layers

Pooling layers reduce the **spatial dimensions** of the feature maps while retaining the most important information.

For example, **max pooling** selects the highest value within a small region. This helps:

- Reduce computational requirements.
- Reduce the number of features that need to be processed.
- Make the network less sensitive to small changes in the position of features.

This is useful for medical images because the important abnormal region may not always occur at exactly the same location.

Fully Connected Layers

After the convolution and pooling stages, the extracted feature maps are flattened into a one-dimensional vector. The fully connected layers then use these learned features to make the final classification.

In this case, the network uses the extracted X-ray features to determine whether the image belongs to the **normal** or **disease-affected** category.

Summary

Thus, the CNN follows a progressive learning process:

Image → Feature Extraction → Important Feature Reduction → Feature Combination → Classification

The convolutional layers and filters learn visual patterns, pooling reduces the feature-map size, and the fully connected layers use the extracted information to make the final prediction. This makes CNNs appropriate for automatically analyzing the large collection of X-ray images described in the case study.

3. Parameter Sharing in CNNs

Parameter sharing is one of the important features of a CNN. Instead of assigning a separate weight to every connection between every input pixel and every neuron, a CNN uses the **same filter weights across different regions of an image**.

For example, consider a **224 × 224** X-ray image and a **3 × 3** convolutional filter. The same 3 × 3 filter is moved across the entire image. Therefore, the filter uses the same **9 weights** at every location, rather than learning a different set of weights for every image position.

For example:

Without parameter sharing:

Each image location → different weights

With parameter sharing:

Same 3 × 3 filter → scanned across the entire image

This greatly reduces the number of parameters that the network needs to learn.

Comparison with a Fully Connected Network

In a fully connected network, every neuron in one layer is connected to **every pixel or neuron in the previous layer**. For a large image such as a chest X-ray, this can result in a very large number of trainable weights.

In contrast, a CNN uses small filters and reuses their weights across the image. Therefore, the number of parameters is much smaller.

For example, a **3 × 3 filter** has:

$3 \times 3 = 9$

weights, plus one bias.

Even though this filter can scan thousands of different locations in the X-ray, the same 9 weights are reused at each location.

Advantages of Parameter Sharing

Parameter sharing provides several important benefits:

1. **Fewer trainable parameters** – This makes the CNN much smaller than an equivalent fully connected network.
2. **Lower computational requirements** – Fewer parameters need to be stored and updated during training.
3. **Faster training** – The network has fewer weights to optimize.

4. **Better feature detection** – A feature learned in one part of an X-ray can also be detected when it appears in another part of the image.
5. **Reduced risk of overfitting** – Having fewer parameters can help the model generalize better to unseen X-ray images.

Therefore, parameter sharing makes CNNs particularly suitable for medical image classification, where images are large and important features can occur at different locations.

4. Regularization Techniques

Overfitting occurs when a CNN learns the training X-ray images too closely, including patterns that may not be useful for classification. As a result, the model may perform well on training images but poorly on new, unseen X-rays.

The following techniques can be used to reduce this problem.

1. Dropout

Dropout randomly deactivates a percentage of neurons during training. For example, a dropout rate of **0.5** means that approximately half of the selected neurons are temporarily ignored during each training iteration.

This prevents the network from becoming overly dependent on particular neurons and encourages it to learn more general features.

For the proposed X-ray classifier, dropout can be placed before or between the fully connected layers.

2. Data Augmentation

Data augmentation creates slightly modified versions of existing X-ray images during training. This can include operations such as:

- Small rotations
- Horizontal shifting
- Zooming
- Brightness adjustments
- Small translations

This effectively increases the variety of training examples without requiring the healthcare organization to collect completely new images.

However, augmentation should be medically appropriate. Transformations that could change the interpretation of an X-ray should be avoided.

3. Batch Normalization

Batch normalization normalizes the activations produced by a layer during training. It helps maintain more stable values as information passes through the network.

This can make CNN training more stable and can also provide a regularizing effect that helps reduce overfitting.

Combined Approach

A suitable strategy for this medical image classifier would therefore be:

X-ray Images → Data Augmentation → Convolution + Batch Normalization → ReLU → Pooling → Deeper CNN Layers → Dropout → Fully Connected Layer → Output

Using these techniques together can help the CNN **generalize better to new chest X-ray images** instead of simply memorizing the training dataset.

5. AlexNet vs ResNet

Both AlexNet and ResNet are CNN architectures that can be used for image classification, but they differ significantly in their depth and training approach.

Feature	AlexNet	ResNet
Architecture	Relatively simple CNN	Deeper CNN using residual blocks
Network depth	8 learnable layers	Can have 18, 34, 50, 101+ layers
Feature learning	Learns basic to complex features	Learns more complex and detailed features
Training	Relatively easier	Deeper networks can be trained effectively using skip connections
Computational requirement	Lower	Generally higher
Classification capability	Good	Generally better for complex image tasks

Why ResNet is More Suitable

For the proposed medical image classification system, **ResNet would be the more appropriate choice**.

Chest X-rays can contain subtle patterns that distinguish a healthy image from a disease-affected image. A deeper network such as ResNet can learn increasingly complex features from the images. Its **residual or skip connections** also help information and gradients pass through deeper layers, making the network easier to train than a conventional very deep CNN.

ResNet can therefore provide stronger feature learning and potentially better classification performance when sufficient training data and computational resources are available.

Computational Consideration

The main disadvantage of ResNet is that deeper architectures generally require **more computational resources and training time** than AlexNet. However, the case study states that the dataset contains **thousands of X-ray images**, making a deeper architecture practical if appropriate hardware is available.

Final Choice

ResNet is recommended for this application because its deeper architecture provides stronger feature-learning capability and its residual connections make deep network training more effective. Although it requires greater computational resources than AlexNet, the potential improvement in classification performance makes it a better choice for automated chest X-ray analysis.

Conclusion:
ResNet > AlexNet for this medical image classification application, particularly when accuracy and detailed feature learning are prioritized over minimum computational cost.

CASE STUDY 2

1. Designing a CNN Architecture for Autonomous Vehicles

A suitable CNN architecture can be designed to process camera images and progressively extract visual features before classifying the objects present in the scene.

A basic architecture would be:

Camera Image
↓
Input Layer
↓
Convolution + ReLU
↓
Pooling
↓
Convolution + ReLU
↓
Pooling
↓
Deeper Convolution Layers
↓
Flatten / Feature Representation
↓
Fully Connected / Output Layer
↓
Object Classes

Input Layer

Images captured by the vehicle's cameras are first resized to a fixed resolution and normalized. This ensures that images received under different conditions can be processed consistently.

Convolutional Layers

The convolutional layers extract visual features from the road scene. Early layers can learn simple features such as **edges, lines, corners, and textures**.

As the image passes through deeper layers, the CNN combines these basic features to learn more complex patterns associated with objects such as:

- Cars
- Pedestrians
- Bicycles
- Traffic signs
- Road obstacles

Pooling Layers

Pooling reduces the spatial dimensions of the feature maps while retaining important information. This decreases computational requirements and helps the network handle small changes in the position of objects.

This is particularly useful in autonomous vehicles because objects can appear at different positions and distances in camera images.

Fully Connected / Output Layer

After feature extraction, the learned features are passed to the final classification stage. The output layer can determine which object categories are present in the image.

For example:

Input: Road camera image
Output: Car + Pedestrian + Traffic Sign

Handling Real-World Conditions

Since the vehicle must operate under **different lighting conditions, viewing angles, distances, and road environments**, the CNN should be trained using a diverse collection of images representing these conditions.

Regularization techniques can also be incorporated to improve generalization and reduce overfitting.

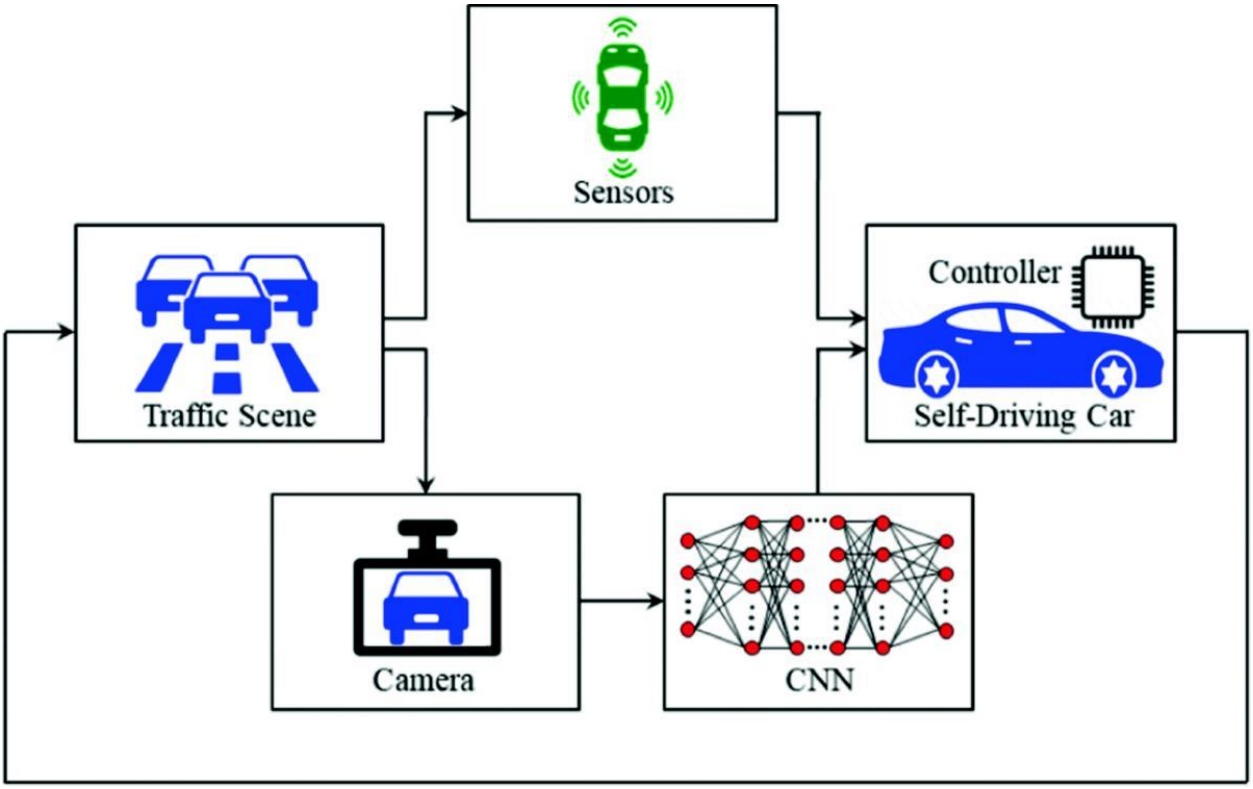
Recommended Architecture

For this application, a **ResNet-based CNN** would be a strong choice because the system needs to learn increasingly complex visual features from complicated road scenes. The case study specifically identifies **AlexNet and ResNet** as the architectures being considered.

Therefore, the overall system can use:

Camera → Preprocessing → ResNet CNN → Feature Extraction → Classification → Detected Objects

This architecture provides a structured way for the autonomous vehicle to extract visual information and recognize important objects in its surroundings.


Sensors
Traffic Scene
Controller
Self-Driving Car
Camera
CNN

2. Role of Different CNN Layers

A CNN processes an image through several stages, with each stage extracting increasingly useful information from the road scene.

1. Input Layer

The input layer receives the image captured by the vehicle's camera. Since camera images may have different dimensions, they can be resized to a standard size before being given to the CNN.

The input contains pixel information representing the **road, vehicles, pedestrians, signs, bicycles, and other objects**.

2. Convolutional Layer

The convolutional layer applies multiple filters to the input image. Each filter scans small regions of the image and detects particular visual patterns.

The first convolutional layers generally learn **low-level features**, such as:

- Edges
- Lines
- Corners
- Simple textures

As the image passes through deeper convolutional layers, these features are combined to form **higher-level features**, such as shapes and object parts.

For example:

Pixels → Edges → Shapes → Object Parts → Complete Objects

This progressive feature learning allows the CNN to recognize different objects in complex road scenes.

3. Activation Layer

After convolution, an activation function such as **ReLU (Rectified Linear Unit)** is applied.

ReLU introduces non-linearity into the network, allowing it to learn complex relationships between the features. Without activation functions, stacking multiple convolutional layers would provide limited additional learning capability.

4. Pooling Layer

Pooling reduces the spatial dimensions of the feature maps while retaining important information.

For example, **max pooling** selects the strongest activation within a small region.

This helps:

- Reduce computational requirements.
- Reduce the size of feature maps.
- Make feature detection less sensitive to small changes in object position.

This is useful in autonomous vehicles because a pedestrian or car can appear at different locations within the camera's view.

5. Fully Connected / Output Layer

After several convolution and pooling stages, the extracted features are passed to the final classification stage.

The fully connected layers combine the learned features and use them to determine the object category.

The output could identify classes such as:

Car | Pedestrian | Bicycle | Traffic Sign | Road Obstacle

Progressive Feature Learning

The major advantage of multiple convolutional layers is that the CNN does not try to recognize an entire object immediately. Instead, it builds its understanding gradually.

Low-level features
Edges, lines and textures
↓
Intermediate features
Shapes and object parts
↓
High-level features
Cars, pedestrians, bicycles and signs
↓
Output
Recognized object classes

Thus, convolutional filters progressively transform raw camera pixels into meaningful high-level representations that can be used for object recognition. This is particularly important for the autonomous vehicle because it must recognize objects under different viewing conditions and environments.

3. Importance of Parameter Sharing

In a CNN, **parameter sharing** means that the same convolutional filter is reused at different locations of an image. Instead of learning separate weights for every pixel location, the CNN learns one set of filter weights and applies it across the entire image.

For example, a **3 × 3 filter** contains only:

$3 \times 3 = 9$

weights, apart from its bias. The same filter can then scan across thousands of locations in a road image.

Reduction in Computational Complexity

A fully connected network would require every pixel to be connected to many neurons, resulting in a very large number of trainable parameters.

CNNs avoid this by using small filters and sharing their weights across the image. This results in:

- Fewer trainable parameters.
- Lower memory requirements.
- Reduced computational complexity.
- Faster and more efficient training.

This is particularly important for autonomous vehicles because their vision systems need to process camera images rapidly.

Translation-Related Feature Detection

Parameter sharing also allows a CNN to recognize the **same feature regardless of where it appears in the image**.

For example, suppose a filter learns to detect the shape of a traffic sign. Because that filter is applied across the entire image, it can detect the sign whether it appears:

Left side of the image → Center → Right side of the image

Similarly, a filter trained to detect a particular edge or shape can identify that feature at different positions in the camera frame.

This is known as **translation-related feature detection** and is highly useful in autonomous vehicles because cars, pedestrians, bicycles, and signs can appear at different locations in every camera frame.

Conclusion

Therefore, parameter sharing allows CNNs to **reuse learned features across different image locations**, reducing computational complexity while making the network effective at detecting objects even when their positions change. This makes it especially valuable for real-time autonomous vehicle vision systems.

4. Overfitting and Regularization

Overfitting occurs when the CNN learns the training images too closely instead of learning general features that work on new images. In an autonomous vehicle, this can be dangerous because the system must recognize objects that it has not encountered during training.

For example, a model trained mostly on **bright, clear daytime images** might perform poorly when it encounters a pedestrian at night or a traffic sign under different lighting.

The following regularization methods can be used:

1. Data Augmentation

Data augmentation creates modified versions of training images to expose the CNN to a wider range of conditions.

Suitable transformations include:

- Small rotations
- Cropping
- Scaling and zooming
- Brightness changes
- Horizontal shifting
- Slight changes in contrast

This helps the model become more robust to different **lighting conditions, viewing angles and object positions**.

2. Dropout

Dropout randomly deactivates some neurons during training. This prevents the CNN from becoming overly dependent on particular neurons.

As a result, the network is encouraged to learn more general features that can be used on previously unseen road images.

3. Batch Normalization

Batch normalization normalizes the activations within the network during training. It helps make the training process more stable and can also provide a regularizing effect.

This can help the CNN generalize better to new camera images.

4. Early Stopping

The model can be monitored using validation data during training. If the validation performance stops improving while training performance continues to improve, training can be stopped.

This prevents the model from continuing to memorize the training dataset.

Conclusion

A combination of **data augmentation, dropout, batch normalization, and early stopping** can reduce overfitting. These techniques help the CNN learn general visual features rather than memorizing particular road images, improving its ability to recognize objects in different real-world environments.

5. AlexNet vs ResNet

Feature	AlexNet	ResNet
Architecture	Conventional CNN with convolution, pooling and fully connected layers	CNN built using residual blocks and skip connections
Depth	Relatively shallow	Can be much deeper
Parameter efficiency	Has a large number of parameters, particularly in fully connected layers	Residual designs can provide better parameter efficiency in deeper networks
Training difficulty	Comparatively easier to train because of its smaller depth	Deep networks are more difficult to train, but skip connections make training easier
Vanishing gradients	More likely to experience gradient problems as networks become deeper	Skip connections help reduce the vanishing-gradient problem
Feature learning	Learns useful low-level and high-level image features	Deeper structure allows learning of more complex and detailed features

AlexNet

AlexNet is an early and influential deep CNN architecture. It uses convolutional layers to extract image features followed by fully connected layers for classification.

Its relatively smaller depth makes it easier to understand and train. However, compared with modern deeper architectures, its ability to learn highly complex features is more limited.

ResNet

ResNet, or **Residual Network**, introduces **skip connections**. Instead of forcing every layer to learn a completely new representation, a residual block allows information to bypass one or more layers.

This helps information and gradients flow through the network and makes it possible to train much deeper CNNs.

Because of its greater depth, ResNet can progressively learn more complex features from road images, which is useful for recognizing objects such as **cars, pedestrians, bicycles, traffic signs, and obstacles**.

Vanishing-Gradient Problem

As conventional neural networks become deeper, gradients can become extremely small while being propagated backward through the network. This is known as the **vanishing-gradient problem** and can make deep networks difficult to train.

ResNet addresses this using skip connections, allowing gradients to travel more directly through the network.

Overall Comparison

For a complex autonomous vehicle vision system, **ResNet provides stronger feature-learning capability and better support for deep networks**, while AlexNet has the advantage of being simpler and generally less computationally demanding.

Therefore, although AlexNet may be suitable for simpler image-classification tasks, ResNet is generally better suited to the complex feature-learning requirements of autonomous vehicle object recognition.